UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DSIC

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Dpto. de Sistemas Informáticos y Computación

Tipos Cociente en un Lenguaje Dependientemente Tipado

Trabajo Fin de Máster

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

AUTOR: Vegas, Luis

Tutor: Escobar Román, Santiago

CURSO ACADÉMICO: 2025-2026

Índice general

1. Introducción **1**
- 1.1. Motivación . . . 1
- 1.2. Objetivos . . . 1
- 1.3. Estructura del la tesis . . . 1

2. Fundamentos teóricos **2**
- 2.1. El Lambda cálculo . . . 2
  - 2.1.1. β-reducción . . . 3
  - 2.1.2. Variables libres y ligadas . . . 3
  - 2.1.3. Sustitución . . . 4
  - 2.1.4. Confluencia . . . 5
- 2.2. Teoria de tipos . . . 5
- 2.3. Lambda cálculo simplemente tipado . . . 6
- 2.4. Tipos dependientes . . . 6
- 2.5. Teoria de tipos de Martin-Löf . . . 6
- 2.6. Reescritura de terminos en Maude . . . 6
- 2.7. Tipos Cociente . . . 6

3. Un lenguaje dependientemente tipado **7**
- 3.1. Sintaxis . . . 7
- 3.2. Sustitucion . . . 7
- 3.3. Normalizacion . . . 7
- 3.4. Verificacion de tipos . . . 7

4. Extension con tipos cociente **8**
- 4.1. Formacion . . . 8
- 4.2. Introduccion . . . 8
- 4.3. Eliminacion . . . 8
- 4.4. Computacion . . . 8

5. Conclusiones **9**

A. Código Maude **10**

Capítulo 1

Introducción

1.1. Motivación

1.2. Objetivos

1.3. Estructura del la tesis

Capítulo 2

Fundamentos teóricos

2.1. El Lambda cálculo

El lambda cálculo fue inventado por Alonzo Church en la decada de los 80 con el proposito de capturar la nocion abstracta de la aplicabilidad de una funcion, sin embargo, conviene precisar en qué sentido. Una función puede entenderse de dos maneras: de forma extensional, como el conjunto de pares entrada–salida que la determina, o de forma intensional, como la regla o el proceso de cómputo que, dado un argumento, prescribe cómo obtener el resultado (Geuvers, 2014). Es esta segunda lectura, la de la función como algoritmo, la que el lambda cálculo formaliza (Barendregt, 1984; Geuvers, 2014). En su origen, Church concibió el sistema como parte de una fundamentación de la lógica y las matemáticas; dicha formulación resultó ser inconsistente, tal y como evidenciaron Kleene y Rosser, por lo que Church aisló su núcleo puramente computacional, libre de constantes lógicas, dando lugar al lambda cálculo tal y como se conoce hoy (Barendregt, 1984), siendo ampliamente adoptado como un sistema formal de lógica matemática.

Este sistema ocupa, además, un lugar cenral el diseño de lenguajes de programación. De forma análoga, la manera en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en dia. De hecho, lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y LISP, estan ampliamente inspirados en un conjunto del lambda calculus y, pueden entenderse, en buena medida, como extensiones dotadas de azucar sintactico, sistemas de tipos y otras primitivas (Pierce, 2002).

El lambda calculo es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicacion y abstracción lambda. De forma inductiva, fijado un conjunto infinito numerable de variables V, el conjunto Λ de los términos lambda se define como el menor conjunto cerrado bajo las siguientes reglas (Barendregt, 1984; Pierce, 2002):

$$
\begin{aligned}
M := \quad & x && (1)\\
\mid \quad & \lambda x.\, M && (2)\\
\mid \quad & M\ N && (3)
\end{aligned}
$$

Donde, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$ y $V \in \Lambda$, entonces $V \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) Es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

A continuación se detallan propiedades de sumo interés que articulan, posteriormente, los fundamentos del lenguaje que describe esta tésis: la distinción entre variables libres y ligadas, junto con la α-equivalencia que de ella se deriva; la operación de sustitución que evita la captura de variables; y la β-reducción como noción primaria de cómputo. Se mencionará igualmente la η-conversión, que captura la extensionalidad de las funciones.

2.1.1. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\to_\beta$ es empleada. La $\to_\beta$ está basada en la *sustitución*, formalmente expresada como $M[x := N]$, significando que M es un término donde x ha sido reemplazado por N.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como $M[\, x := N]$, donde toda ocurrencia de x en M (estando x libre en M), debe ser reemplazada por N (Geuvers, 2014).

Ejemplo 2.1. $(\lambda x.\, x + 1)\, 5$ reduce a $(x + 1)[x := 5] = 5 + 1 = 6$.

Ejemplo 2.2. La función identidad aplicada a un término cualquiera devuelve dicho término: $(\lambda x.\, x)\, y \to_\beta x[x := y] = y$.

Ejemplo 2.3. La β-reducción puede encadenarse, reduciendo paso a paso hasta que no queden más redexes. considérese el combinador $k \equiv \lambda x.\, \lambda y.\, x$ aplicado a dos argumentos:

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\, a\, b &\to_\beta (\lambda y.\, x)[x := a]\, b \\ &= (\lambda y.\, a)\, b \\ &\to_\beta a[y := b] = a.\end{aligned}$$

Nótese que el segundo argumento, b, es descartado, pues y no ocurre en el cuerpo.

2.1.2. Variables libres y ligadas

La abstracción $\lambda x.\, M$ introduce un *binder* sobre la variable x, cuyo *alcance* es el cuerpo M. Decimos, pues, que una ocurrencia de una variable x está *ligada* cuando aparece dentro del alcance de un λx que la introduce; en caso contrario, dicha ocurrencia se dice *libre*.

Conviene resaltar que una misma variable puede ocurrir libre y ligada dentro de un mismo término. En $(\lambda x.\, x)\, x$, por ejemplo, la primera ocurrencia de x está ligada por el λx, mientras que la segunda, externa a la abstracción, permanece libre. Formalmente, el conjunto de variables libres de un término, denotado $FV(M)$, se define de forma inductiva sobre la estructura del término (Pierce, 2002):

$$\begin{aligned}&(1)\ FV(x) = \{x\}, \\ &(2)\ FV(\lambda x.\, M) = FV(M) \setminus \{x\}, \\ &(3)\ FV(M\, N) = FV(M) \cup FV(N).\end{aligned}$$

(1) indica que una variable, por sí sola, es libre. (2) establece que al abstraer sobre x, esta deja de ser libre en $\lambda x.\, M$, pues queda ligada por el nuevo binder; por ello se retira del conjunto. (3) recoge que las variables libres de una aplicación son, simplemente, la unión de las libres de cada subtérmino. Un término M tal que $FV(M) = \emptyset$ se denomina

cerrado (closed) o *combinador*.

Por último, el nombre concreto de una variable ligada carece de relevancia: lo único que importa es la estructura de ligaduras que establece. Así, $\lambda x.\, x$ y $\lambda y.\, y$ denotan exactamente la misma función identidad. A esta noción, que identifica términos que difieren únicamente en el nombre de sus variables ligadas, se le conoce como α-*equivalencia*, y se denota $=_\alpha$. El proceso de renombrar una variable ligada para obtener un término α-equivalente recibe el nombre de α-*conversión*, y será fundamental para definir correctamente la sustitución bajo un binder.

2.1.3. Sustitución

Anteriormente, definimos la sustitución en términos de $\rightarrow_\beta$ reducción, sin embargo, la sustitucion posee en si reglas de computo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto del todo.

Se definen, pues, las reglas de computo de la sustitucion de acuerdo a lo discutido por Pierce (2002). Más formalmente:

(1) $x[x := N] \equiv N$,
(2) $y[x := N] \equiv y$ if $x \not\equiv y$,
(3) $(PQ)[x := N] \equiv (P[x := N])(Q[x := N])$,
(4) $(\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y \rightarrow z}[x := N])$, if $\lambda z.\, P^{y \rightarrow z}$ is an α-variant of $\lambda y.\, P$ such that $z \notin FV(N)$.

Para (1), sustituyendo x, por N dentro de x, siendo x el termino que buscamos sustituir, obtenemos N. De forma analoga, sustituyendo cada ocurrencia de x en el termino y, por N, si x e y no son iguales, el termino intacto, y, sera retornado. El caso de (3), tiene sentido cuando P es una abstraccion aplicada a un termino Q. Concretamente, se reduce a la sustitucion de x por N, en ambos P y Q. (4) contempla la sustitución bajo un binder. Lo que se busca es sustituir x por N dentro del cuerpo P de la abstracción $\lambda y.\, P$. Sin embargo, si procedíeramos de forma ingenua y N contuviera a y como variable libre, dicha y quedaría inadvertidamente atrapada por el ligador λy, alterando por completo el significado del término. A este fenómeno se le conoce como *captura de variables*.

Para evitarlo, antes de sustituir se renombra la variable ligada y por una variable fresca z, mediante α-conversión, escogida de modo que $z \notin FV(N)$ (y, por extensión, que no genere conflicto con las variables ya presentes en P). Solo entonces se procede a sustituir x por N dentro del cuerpo ya renombrado. De esta manera, la sustitución preserva el significado del término original, razón por la cual se la describe como una sustitución *que evita la captura* (Pierce, 2002).

Ejemplo 2.4. Supóngase la sustitución $(\lambda y.\, x)[x := y]$. Si sustituyéramos directamente, obtendríamos $\lambda y.\, y$, es decir, la identidad: la y libre que introdujimos habría sido capturada por el binder. Renombrando primero la variable ligada a una fresca z, obtenemos $(\lambda z.\, x)[x := y] = \lambda z.\, y$, que es el resultado correcto, pues la y permanece libre.

2.1.4. Confluencia

Anteriormente mencionamos como la beta-reduccion emula, de cierta manera, el proceso de computacion de una expresion dada: evaluado un termino, es seguro obtener la forma normal del mismo, es decir, no existe un redex en la expression que permita seguir reduciendola.
El orden en que se evaluen estos redexes es importante: los mismos pueden influir en si el computo termina o, mejor dicho, alcanza una forma normal concreta. Por tanto, dado un termino t, y dos terminos $\rightarrow_\beta t_1$ y $\rightarrow_\beta t_2$ que derivan del mismo t en uno o varios pasos de beta-reduccion, ¿es posible llegar a un termino comun u alcanzable desde t_1 y t_2? Si existe tal u, se dice, por tanto, que t es confluente.

En el presente capitulo, describimos esta propiedad con detenimiento, y estudiaremos (?) como la beta reduccion es confluente.

La confluencia, tambien conocida como el teorema de Church-Rosser, es una propiedad de los sistemas de reescritura que describe cuales terminos en un sistema puede ser reescrito (reducido en uno o más pasos) para producir el mismo resultado. La importancia del teorema redica en que el resultado del computo, debe ser independiente del orden en que se ejecutaron los pasos de reduccion. Formalmente, se describe como sigue:

Teorema 2.1. *(Church-Rosser) – Citar: Barendregt, 1981, p. 62 Dado un termino λM, tenemos que $M \rightarrow_\beta^* N_1$ y $M \rightarrow_\beta^* N_2$, entonces existe un termino λN_3 tal que $N_1 \rightarrow_\beta^* N_3$ y $N_2 \rightarrow_\beta^* N_3$.*

Lema 2.2. *1. M es una forma normal si M no contiene mas redexes.*

2. M tiene una forma normal si existe un N, en forma normal, tal que $M =_\beta N$. Tal N es la forma normal de M.

La confluencia es representada por el siguiente diagrama: –diagrama aqui–

Ejemplo 2.5. Ejemplo de diamante con una lambda exprsion

De aqui, se desprende el problemas como la terminacion. La terminacion, determina cuando el computo de una funcion termina, o no. En el contexto del lambda calculo, un redex puede no terminar de reducirse o bien, puede entrar en un ciclo infinito de pasos de reduccion, siendo imposible alcanzar una forma normal.

Ejemplo 2.6. Ejemplo de diamante con una lambda exprsion no terminante

Aun asi, es importante notar que el lambda calculo es terminante por beta-reduccion.

Ejemplo 2.7. insertar prueba

2.2. Teoria de tipos

Los tipos, pese a que son similares a los conjuntos, son ligeramente distintos. Por conjunto, lo entendemos con una coleccion de elementos, similares, o no, que poseen una propiedad concreta, de donde es posible extenderlos y razonar sobre mas relaciones

de pertenencia y formar, por ejemplo, conjuntos dentro de otros conjuntos. Los tipos, a diferencia de los conjuntos que dan informacion semantica, proveen informacion sintactica de los elementos. Usualmente en la teoria de conjuntos definimos que un elemento forma parte de un junto, como sigue:

$$4 \in \{\, n \in \mathbb{N} | x > 5 \,\} \tag{2.1}$$

En la teoria de tipos, las relaciones de pertenencia:

$$3 + 1 : \mathbb{N} \tag{2.2}$$

Esto puede interpretarse, desde el lente de teoria de tipos, como 3 + 1 es un termino en los naturales, o bien, de tipo N.

La nocion de tipos es atribuida a Bertrand Russell, quien desarrollo la teoria de tipos en respuesta a las contradicciones que, para entonces, en la teoria de conjuntos y la logica formal existian, como la paradoja de Russell, donde era posible definir, por ejemplo, el conjunto que contiene a todos los conjuntos que no son miembros de si mismo:

$$R = \{\, x | x \in x \,\} \tag{2.3}$$

entonces

$$R \in R \iff R \notin R \tag{2.4}$$

Es, por tanto, de forma contradictoria, un conjunto que es elemento de sí mismo. La solucion para evitar esta paradoja fue emplear una teoria que tuviese una jerarquia de tipos y que, de esta forma, ningún tipo se contuviese a sí mismo.

Los tipos son ubicuos en computacion, pues, numerosos lenguajes tienen su propio sistema de tipos, unos mas avanzados que otros y, otros muchos mas simples. C++ es un claro ejemplo de un sistema de tipos complejo, pero sumamente rico, que facilita posteriormente, una distincion semantica en el lenguaje. Java está dotado de un sistema de tipos ligeramente más robusto que C++,

2.3. Lambda cálculo simplemente tipado

2.4. Tipos dependientes

2.5. Teoria de tipos de Martin-Löf

2.6. Reescritura de terminos en Maude

2.7. Tipos Cociente

Capítulo 3

Un lenguaje dependientemente tipado

3.1. Sintaxis

3.2. Sustitucion

3.3. Normalizacion

3.4. Verificacion de tipos

Capítulo 4

Extension con tipos cociente

4.1. Formacion

4.2. Introduccion

4.3. Eliminacion

4.4. Computacion

Capítulo 5

Conclusiones

Apéndice A

Código Maude

TODO

Bibliografía

Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.

German Geuvers. *Type Theory and Formal Proof*. Cambride University Press, 2014.

Samuel Mimram. *Program = Proof*. Independently published, 2020.

Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.

Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.